A PROJECT REPORT ON · MCA SEMESTER 3 · 2026-27

ChainShield Transportation System

Real-Time Disruption Prediction & Route Optimisation for Logistics

Prepared By (Group No. 59)
Bhavik A. Visani (D51 – 25004401110170)
Diya Shah (D36 – 25003401110017)
Shruti Somani (D42 – 25004401110259)

Guided By
Prof. Dhaval Shah

LJ Institute of Computer Application | Affiliated to LJK University

CONTENTS

Index

1 **Introduction**

Problem Definition · Objective of the New System · Core Components · Project Profile · Need of the New System · Scope of the New System · Advantages and Limitations of the Proposed System · Proposed Time Line Chart · Targeted Users

2 **Requirement Determination and Analysis**

Requirement Determination · Requirement Specification

3 **System Design**

Use Case Diagram · Data Dictionary · User Interface Design · Report Design

4 **Development**

Coding Standards · Implementation Details · Output Screens

5 **Testing**

Test Cases

6 **Proposed Enhancement**

7 **Conclusion**

8 **Bibliography**

1.1 Problem Definition

- Hundreds of shipments move at once across road, rail, sea and air
- Exposed to weather, hub/port congestion, customs holds, breakdowns, sensitive cargo
- Disruptions are noticed only after the delivery date has slipped
- Manual tracking — phone calls, spreadsheets, e-mail
- Late detection → fewer route options, higher recovery cost, lost trust
- No system scores risk continuously and recommends a corrective route in advance

1.2 Objective of the New System

- Continuously analyse weather, congestion, cargo and history for every shipment
- Detect disruptions early with a transparent weighted risk score
- Auto-generate ranked route options: hold, hub reroute, mode switch
- Notify managers and field operators in real time
- Role-based dashboards for Admin, Manager and Field Operator
- Low-cost, self-hosted deployment with Flask + Docker

1.3 Core Components

- Risk Engine — 4-factor score every 60 s
- Disruption Manager — opens at MEDIUM, escalates at HIGH
- Route Solver — hold / hub reroute / mode switch, ranked
- Notifications & Chat — real-time over Socket.IO
- Admin Panel & Reports — users, hubs, cargo, audit log, CSV
- AI Assistant — Gemini with offline fallback
- Live Map — positions, routes, disruption zones

1.4 Project Profile

Particulars	Details
Project Title	ChainShield Transportation System
Group	Group No. 59
Team Members	Bhavik A. Visani (D51) · Diya Shah (D36) · Shruti Somani (D42)
Guide	Prof. Dhaval Shah
Domain	Logistics & Supply Chain Management
Front End	HTML5, CSS3, Bootstrap 5, Jinja2, JavaScript, Leaflet.js, Chart.js
Back End / DB	Python 3.10, Flask, Flask-SQLAlchemy / SQLite
Tools & APIs	Socket.IO, APScheduler, Gemini API, OpenWeatherMap, Docker, Git
Duration	Aug 2026 – Nov 2026 (Semester 3)

1.5 Need of the New System

- Reactive monitoring — problems found after complaints
- Tracking shows location, not upcoming risk
- Weather, congestion, cargo and history data in silos
- Rerouting based on guesswork, not data
- Operators informed late by phone or radio
- No audit trail of decisions

1.6 Scope of the New System

In scope

- Risk scoring of every active shipment
- Auto disruption + ranked route solutions
- Manager, operator and admin dashboards
- Shipment CRUD with validation
- Reports with date filter + CSV export
- AI assistant and live map
- Demo mode for presentations

Out of scope

- Live GPS / telematics hardware
- Payment, billing and SLA modules
- Multi-tenant (many companies)

1.7 Advantages and Limitations of the Proposed System

Advantages

- Preemptive, not reactive
- Transparent weighted scoring
- Operator notified within seconds
- Role-based views, less overload
- Cheap, portable Docker deployment

Limitations

- Near-real-time (60 s interval)
- Public / simulated data feeds
- SQLite — single server only
- AI answers are advisory

1.8 Proposed Time Line Chart

Sr.	Phase	Aug 2026				Sep 2026				Oct 2026				Nov 2026			
		W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
1	Requirement Gathering (R.G.)		■														
2	Requirement Analysis & SRS		■	■													
3	System Design (Use Case, DB, UI)			■	■												
4	Environment & Docker Setup			■													
5	Coding / Development			■	■	■	■	■									
6	UI Enhancement (Themes, Mobile)							■	■								
7	Testing							■	■	■							
8	Implementation / Deployment									■	■						
9	Documentation		■	■	■	■	■	■	■	■	■	■	■	■			
10	Final Review, Submission & Viva												■	■	■		

W1 = days 1–7 · W2 = 8–14 · W3 = 15–21 · W4 = 22 to month-end · Documentation runs through every phase

1.9 Targeted Users

- Administrator — users & roles, hubs, cargo types, config, audit log; all manager rights
- Manager — creates shipments, reviews disruptions, applies route solution, reports, AI
- Field Operators — Driver (road), Loco Pilot (rail), Pilot (air), Captain (sea)
- Operators see only their shipments, update status, get instant route alerts, chat
- Scheduler (system actor) — scans risk every 60 s, opens disruptions, sends alerts
- Indirect — customers, partners and management benefit from fewer delays

2.1 Requirement Determination

- Sources: group discussion, study of manual tracking, analysis of user roles
- Login required for every user
- Continuous, automatic risk evaluation
- Auto disruption at MEDIUM (≥ 50), escalation at HIGH (≥ 70)
- ≥ 3 ranked route alternatives per disruption
- Instant notification to field operators
- Admin management + audit trail; demo mode

2.2 Requirement Specification

Functional

- Login + role-based access
- Shipment CRUD
- Scheduled risk scoring
- Auto disruption alerts
- Ranked route solutions
- Real-time notifications & chat
- Admin CRUD + audit log
- AI assistant, reports + CSV, live map

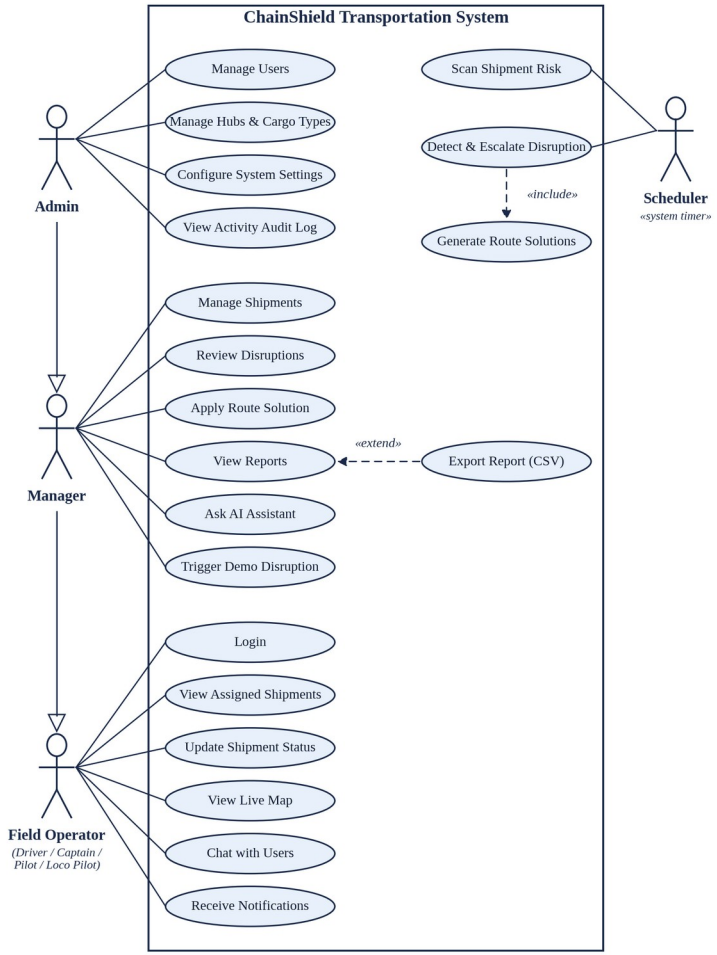
Non-Functional

- Performance — refresh ≤ 60 s
- Usability — responsive, themes
- Portability — Docker
- Reliability — transactional
- Security — hashing, CSRF, roles
- Maintainability — blueprints

Platform

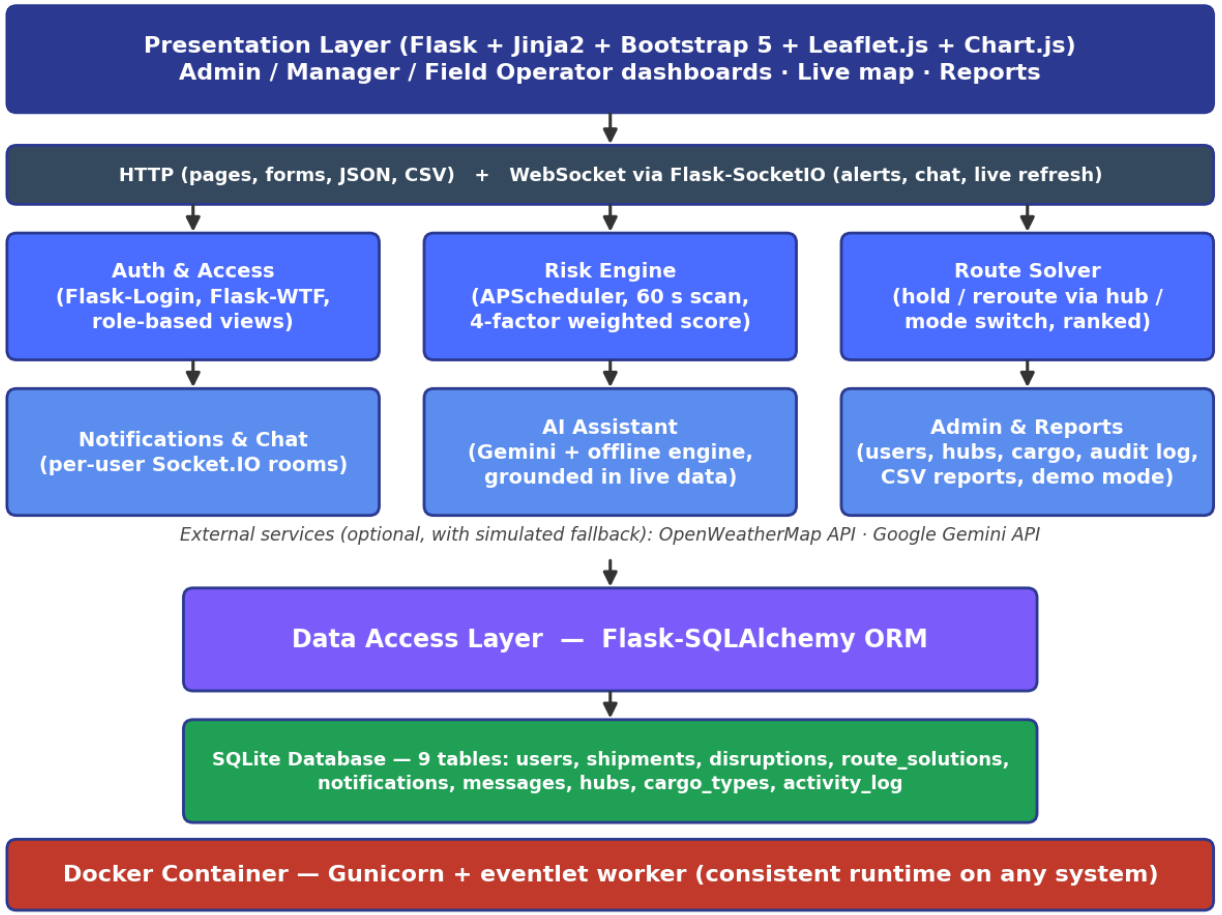
- Python 3.10 + Flask
- SQLite
- Bootstrap 5, Leaflet, Chart.js
- Docker + Gunicorn
- Gemini + OpenWeatherMap
- Windows 11 / any Docker OS

3.1 Use Case Diagram



- Actors: Admin, Manager, Field Operator, Scheduler
- Generalization: Admin → Manager → Field Operator (inherits use cases)
- «include»: Detect & Escalate Disruption → Generate Route Solutions (always)
- «extend»: Export Report (CSV) → View Reports (optional)
- Straight lines, no overlapping

3.1 System Architecture



3.2 Data Dictionary (1/4) — Sample record: id = 1

Table: users

Column	Type	Sample (id = 1)
id	INTEGER	1
name	VARCHAR(100)	Admin
email	VARCHAR(100)	admin@chainshield.com
password_hash	VARCHAR(255)	scrypt:32768:8:1$jCqB…
role	VARCHAR(20)	admin
phone	VARCHAR(20)	9876543210
is_active	BOOLEAN	1
created_at	DATETIME	2026-08-25 03:48:26

Table: hubs

Column	Type	Sample (id = 1)
id	INTEGER	1
name	VARCHAR(100)	Ahmedabad Hub
location	VARCHAR(150)	Ahmedabad, Gujarat
lat	FLOAT	23.0225
lng	FLOAT	72.5714

Table: cargo_types

Column	Type	Sample (id = 1)
id	INTEGER	1
name	VARCHAR(50)	General
sensitivity_weight	FLOAT	0.3

3.2 Data Dictionary (2/4) — Sample record: id = 1

Table: shipments — Every shipment with its route, cargo, assigned operator, live position and current risk.

Column	Data Type	Constraints	Sample (id = 1)
id	INTEGER	PRIMARY KEY, AUTO INCREMENT	1
tracking_no	VARCHAR(30)	NOT NULL, UNIQUE	CS-1001
origin	VARCHAR(100)	NULL	Ahmedabad
destination	VARCHAR(100)	NULL	Mumbai
cargo_type_id	INTEGER	FOREIGN KEY → cargo_types.id	2 (Perishable)
transport_mode	VARCHAR(20)	NULL	Road
status	VARCHAR(20)	DEFAULT 'In-transit'	In-transit
current_lat	FLOAT	NULL	21.1702
current_lng	FLOAT	NULL	72.8311
risk_score	FLOAT	DEFAULT 0.0	55.98
risk_level	VARCHAR(15)	DEFAULT 'LOW'	MEDIUM
cargo_value	FLOAT	DEFAULT 0.0	85000.00
assigned_operator_id	INTEGER	FOREIGN KEY → users.id	8 (Suresh Yadav, driver)
hub_id	INTEGER	FOREIGN KEY → hubs.id	1 (Ahmedabad Hub)
created_at	DATETIME	DEFAULT current UTC time	2026-09-07 06:24:35

3.2 Data Dictionary (3/4) — Sample record: id = 1

Table: disruptions

Column	Type	Sample (id = 1)
id	INTEGER	1
shipment_id	INTEGER	1 (CS-1001)
type	VARCHAR(30)	Congestion
severity	VARCHAR(15)	MEDIUM
risk_score	FLOAT	55.98
status	VARCHAR(15)	Resolved
detected_at	DATETIME	2026-09-24 13:07:46
resolved_at	DATETIME	2026-09-24 13:34:46
resolved_by	INTEGER	5 (Manisha Patel)

Table: route_solutions

Column	Type	Sample (id = 1)
id	INTEGER	1
disruption_id	INTEGER	1
option_no	INTEGER	1
description	TEXT	Reroute via Jaipur Hub (+8 km detour)
extra_time_hours	FLOAT	1.5
extra_cost	FLOAT	1702.72
risk_score	FLOAT	16.73
selected	BOOLEAN	1

3.2 Data Dictionary (4/4) — Sample record: id = 1

Table: notifications

Column	Type	Sample (id = 1)
id	INTEGER	1
user_id	INTEGER	1 (Admin)
message	VARCHAR(255)	MEDIUM risk on CS-1001 (Ahmedabad → Mumbai) — Congestion, score 55.98
is_read	BOOLEAN	1
created_at	DATETIME	2026-09-24 13:07:46

Table: messages

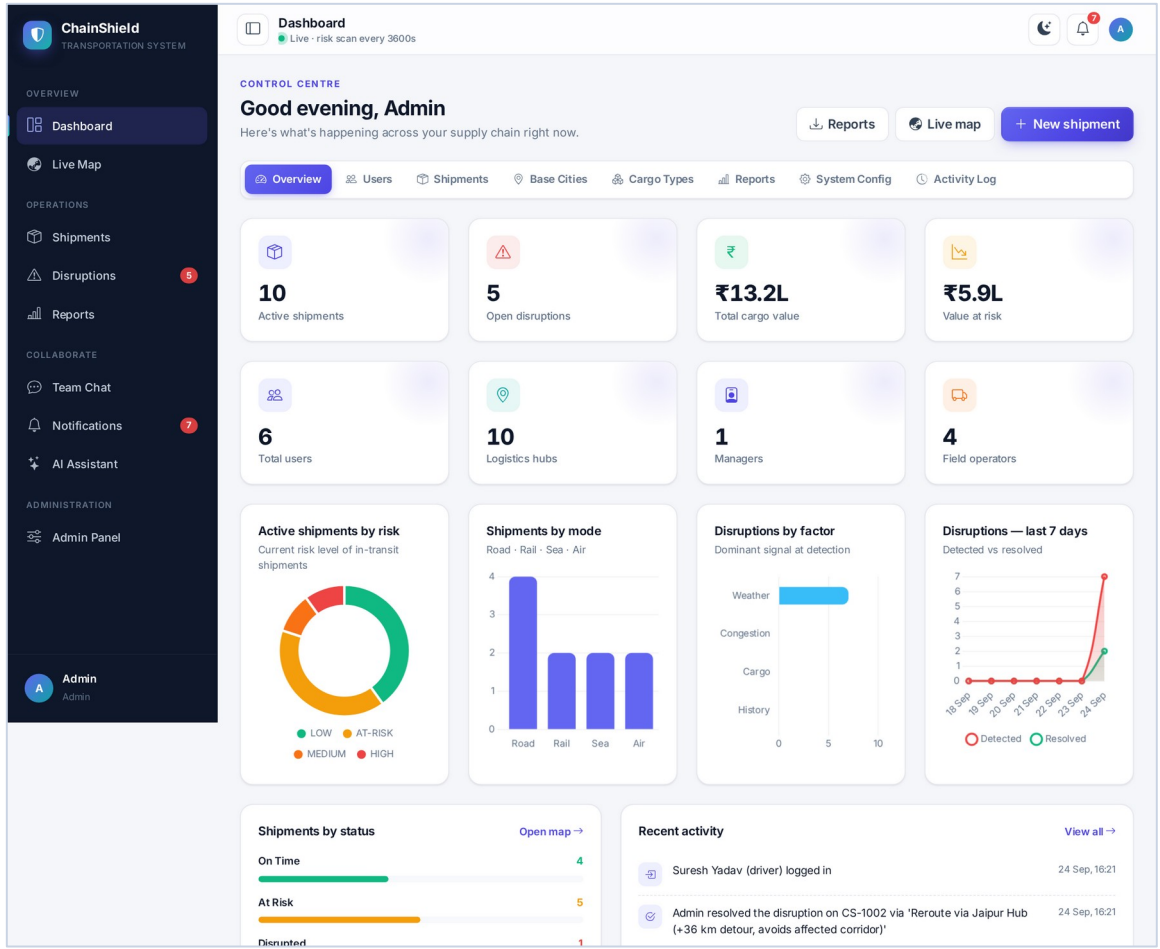
Column	Type	Sample (id = 1)
id	INTEGER	1
sender_id	INTEGER	8 (Suresh Yadav)
receiver_id	INTEGER	1 (Admin)
chat_id	VARCHAR(10)	1-8
content	TEXT	Hello
sent_at	DATETIME	2026-09-24 13:22:49

Table: activity_log

Column	Type	Sample (id = 1)
id	INTEGER	1
user_id	INTEGER	1 (Admin)
action	VARCHAR(255)	Admin resolved the disruption on CS-1002 via 'Hold at current hub'
timestamp	DATETIME	2026-09-07 07:15:49

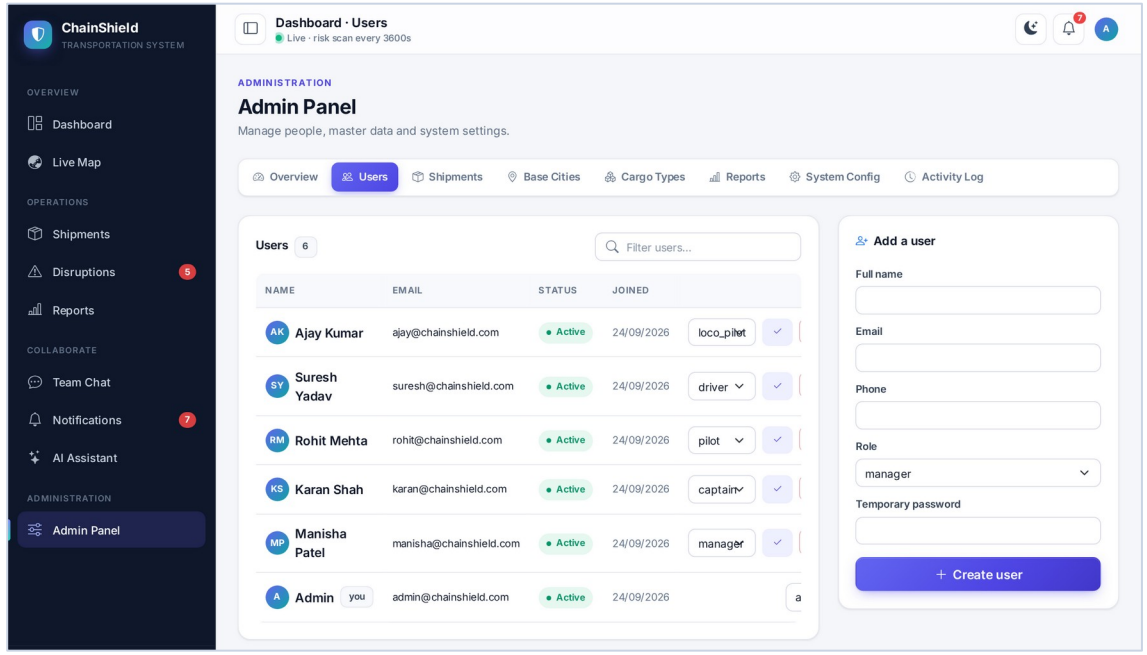
3.3 User Interface Design

- Jinja2 + Bootstrap 5 with a custom design system
- Shared shell: collapsible sidebar, top bar, notification bell
- Fixed risk colours: LOW green, AT-RISK amber, MEDIUM orange, HIGH red
- Light / dark themes, fully responsive
- Admin panel tabs: Overview, Users, Shipments, Base Cities, Cargo, Reports, Config, Activity Log

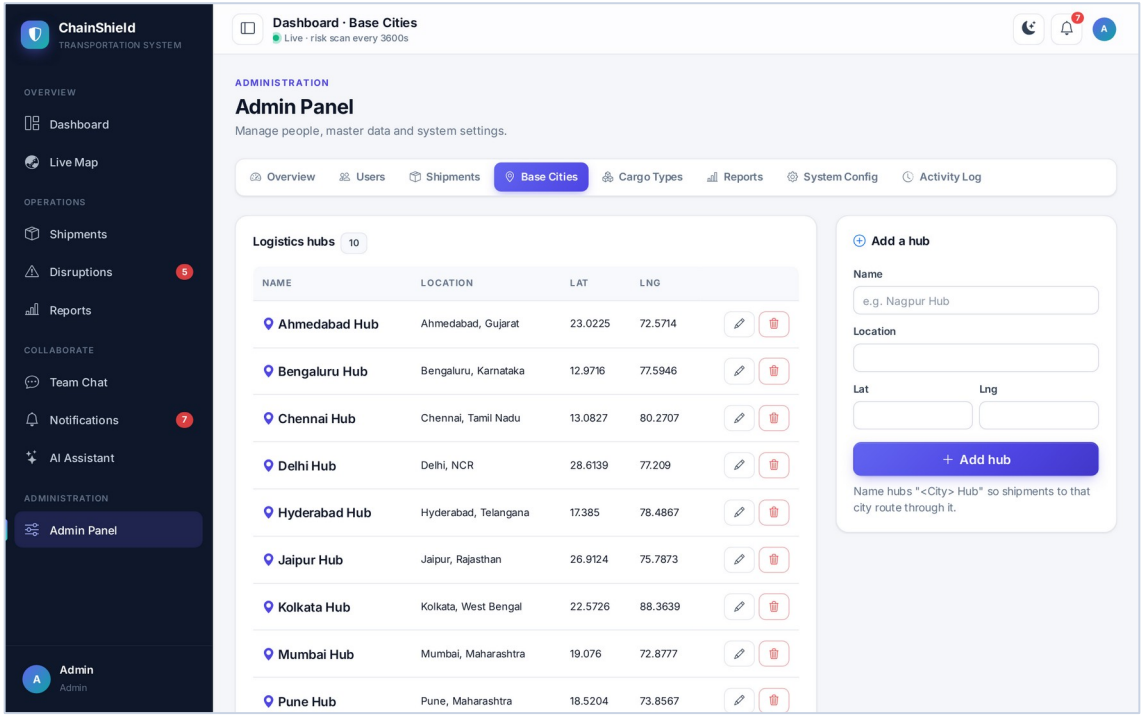


Admin Panel — Overview Dashboard

3.3 User Interface Design — Admin Panel

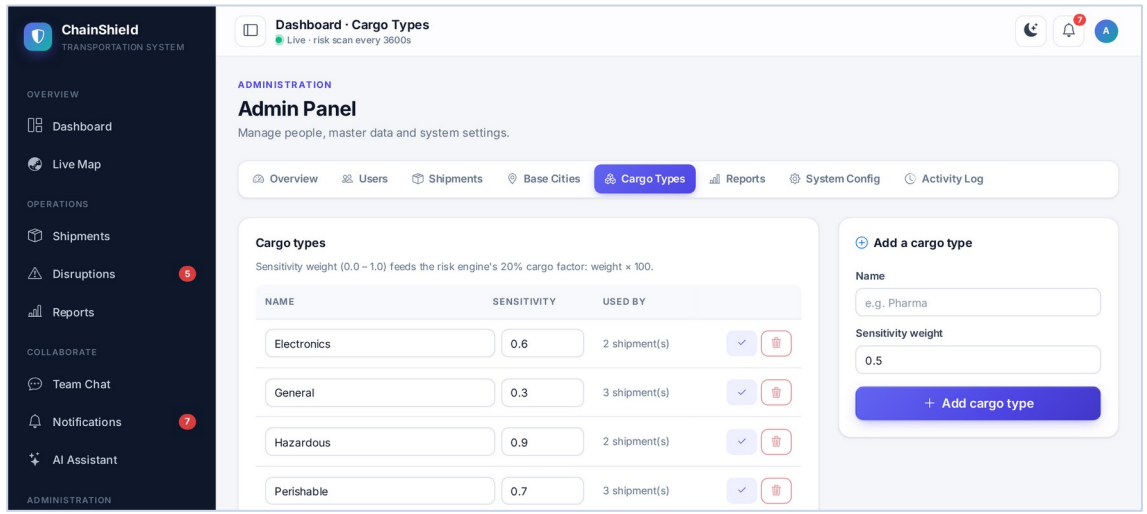


Admin Panel — User Management

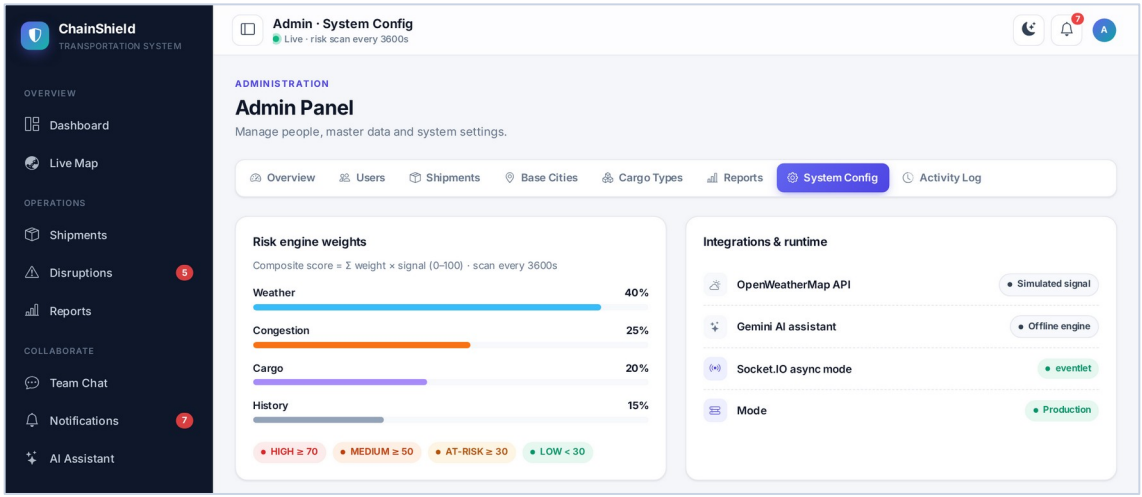


Admin Panel — Logistics Hubs

3.3 User Interface Design — Admin Panel

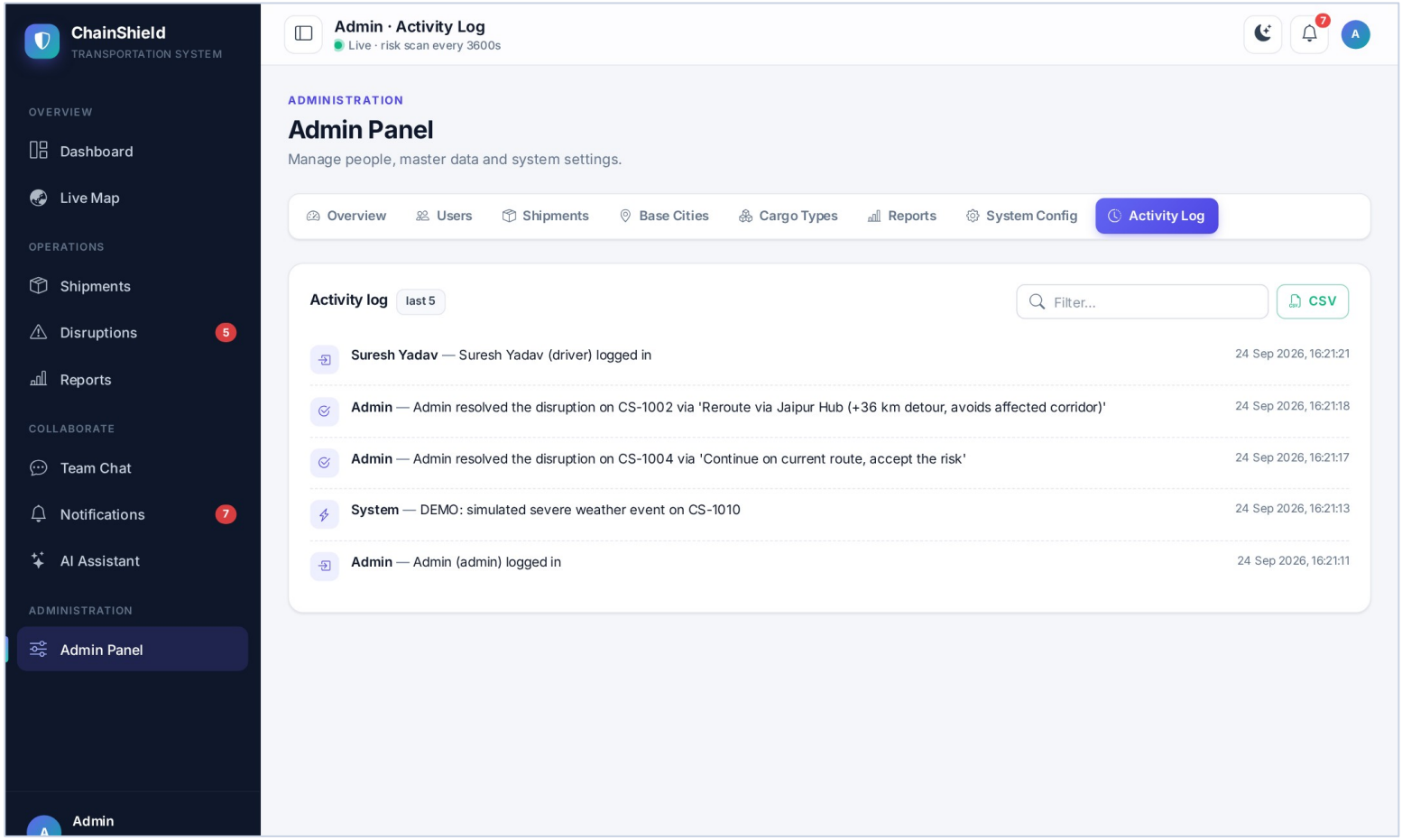


Admin Panel — Cargo Types



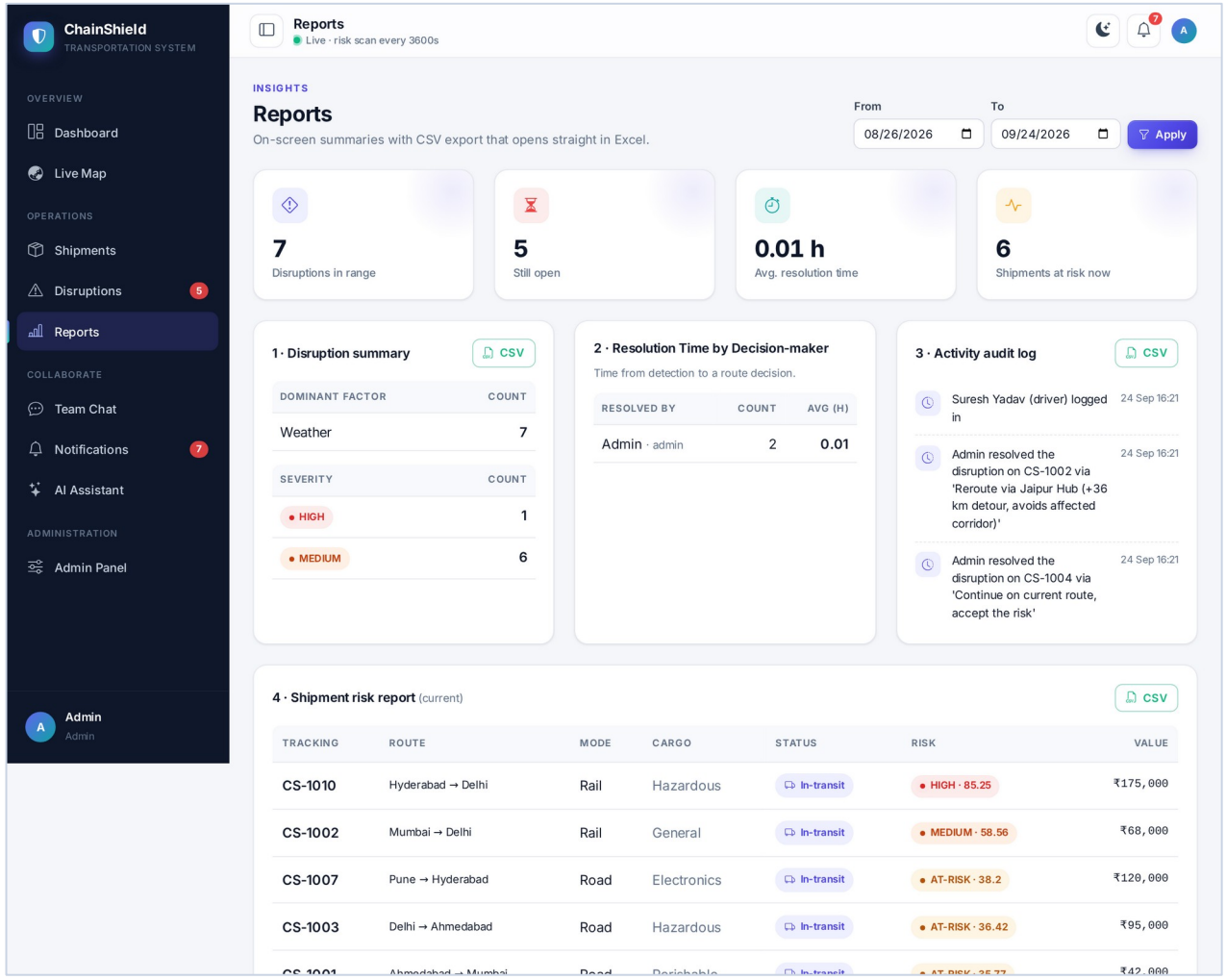
Admin Panel — System Configuration

3.3 User Interface Design — Admin Panel



Admin Panel — Activity Audit Log

3.4 Report Design



- Available to Admin and Manager
- From – To date-range filter
- 1. Disruption Summary (CSV)
- 2. Resolution Time per decision-maker
- 3. Activity Audit Log (CSV)
- 4. Shipment Risk (CSV)
- CSV opens directly in Excel

4.1 Coding Standards (1/9)

1. PEP 8 Naming, Type Hints and Docstrings

- All Python code follows PEP 8: 4-space indentation, snake_case for functions and variables, PascalCase for classes and UPPER_CASE for constants.

- Functions carry type hints and a docstring, so every member of the group can read and review another member's code without explanation.

Example: compute_risk_score() and classify_risk() in app/services/risk_engine.py; the constant ALERT_LEVELS; the class RouteSolution in app/models.py.

chainshield-mca/app/services/risk_engine.py — lines 32-52

```python
def compute_risk_score(weather: float, congestion: float, cargo: float, history: float) -> float:
    """Return a composite 0-100 risk score from the four weighted factors."""
    cfg = current_app.config
    score = (
        weather * cfg["RISK_WEIGHT_WEATHER"]
        + congestion * cfg["RISK_WEIGHT_CONGESTION"]
        + cargo * cfg["RISK_WEIGHT_CARGO"]
        + history * cfg["RISK_WEIGHT_HISTORY"]
    )
    return round(score, 2)


def classify_risk(score: float) -> str:
    if score >= 70:
        return "HIGH"
    if score >= 50:
        return "MEDIUM"
    if score >= 30:
        return "AT-RISK"
    return "LOW"
```

Screenshot — Naming, type hints and docstring in the risk engine

4.1 Coding Standards (2/9)

2. Application Factory Pattern

- The Flask application is never created at import time; create_app() builds it from a named configuration (development, testing or production).
- This lets the same code run on a laptop, inside the automated tests and inside Docker, each with its own settings.

Example: create_app("testing") gives the test suite a fresh in-memory database, while Docker calls create_app("production").

chainshield-mca/app/__init__.py — lines 15-26

```
15 def create_app(config_name=None):
16     config_name = config_name or os.environ.get("FLASK_CONFIG", "development")
17
18     app = Flask(__name__, instance_relative_config=True)
19     app.config.from_object(config_by_name[config_name])
20
21     os.makedirs(app.instance_path, exist_ok=True)
22
23     register_extensions(app)
24     register_blueprints(app)
25     register_cli(app)
```

Screenshot — create_app() in app/__init__.py

4.1 Coding Standards (3/9)

3. Modular Blueprint Structure

- Each feature lives in its own Flask blueprint with its own URL prefix, routes and templates.
- A feature can therefore be changed, tested or disabled without touching the others.

Example: shipments_bp serves everything under /shipments, disruptions_bp under /disruptions and chat_bp under /chat.

chainshield-mca/app/__init__.py — lines 129-144

```python
def register_blueprints(app):
    from app.blueprints.main.routes import main_bp
    from app.blueprints.auth.routes import auth_bp
    from app.blueprints.shipments.routes import shipments_bp
    from app.blueprints.disruptions.routes import disruptions_bp
    from app.blueprints.notifications.routes import notifications_bp
    from app.blueprints.chat.routes import chat_bp
    from app.blueprints.assistant.routes import assistant_bp

    app.register_blueprint(main_bp)
    app.register_blueprint(auth_bp, url_prefix="/auth")
    app.register_blueprint(shipments_bp, url_prefix="/shipments")
    app.register_blueprint(disruptions_bp, url_prefix="/disruptions")
    app.register_blueprint(notifications_bp, url_prefix="/notifications")
    app.register_blueprint(chat_bp, url_prefix="/chat")
    app.register_blueprint(assistant_bp, url_prefix="/assistant")
```

Screenshot — Blueprint registration in app/__init__.py

4.1 Coding Standards (4/9)

4. Single Source of Configuration — No Hard-coded Secrets

- Risk weights, the scan interval and model names are defined once in app/config.py; API keys and the secret key are read from a .env file.
- The .env file is excluded from Git and Docker, so no password or key is ever committed to the repository.

Example: RISK_WEIGHT_WEATHER = 0.40 is read by the risk engine; GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY", "").

chainshield-mca/app/config.py — lines 7-21

```
7  class BaseConfig:
8      SECRET_KEY = os.environ.get("SECRET_KEY", "dev-secret-key-change-me")
9      SQLALCHEMY_TRACK_MODIFICATIONS = False
10     PERMANENT_SESSION_LIFETIME = timedelta(hours=8)
11
12     # Risk engine
13     RISK_SCAN_INTERVAL_SECONDS = int(os.environ.get("RISK_SCAN_INTERVAL_SECONDS", 60))
14     RISK_WEIGHT_WEATHER = 0.40
15     RISK_WEIGHT_CONGESTION = 0.25
16     RISK_WEIGHT_CARGO = 0.20
17     RISK_WEIGHT_HISTORY = 0.15
18
19     # External APIs (optional — services fall back to simulated data if unset)
20     OPENWEATHER_API_KEY = os.environ.get("OPENWEATHER_API_KEY", "")
21     GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY", "")
```

Screenshot — Centralised settings in app/config.py

4.1 Coding Standards (5/9)

5. ORM Models Instead of Raw SQL

- All nine tables are Python classes in app/models.py, and every query goes through Flask-SQLAlchemy.
- Queries are parameterised automatically, which prevents SQL injection, and the database can be moved to PostgreSQL by changing only DATABASE_URL.

Example:
User.query.filter_by(email=form.email.data).first() instead of a hand-written SELECT statement.

chainshield-mca/app/models.py — lines 15-34

```python
class User(db.Model, UserMixin):
    __tablename__ = "users"

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False, index=True)
    password_hash = db.Column(db.String(255), nullable=False)
    role = db.Column(db.String(20), nullable=False, default="manager")
    # roles: admin | manager | captain | pilot | driver | loco_pilot
    phone = db.Column(db.String(20))
    # Overrides UserMixin.is_active: deactivated users can't log in (and are
    # logged out on their next request — see load_user in app/__init__.py).
    is_active = db.Column(db.Boolean, nullable=False, default=True, server_default="1")
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    def set_password(self, raw_password):
        self.password_hash = generate_password_hash(raw_password)

    def check_password(self, raw_password):
        return check_password_hash(self.password_hash, raw_password)
```

Screenshot — The User model with password hashing helpers

4.1 Coding Standards (6/9)

6. Server-side Form Validation

- Every form is validated on the server with Flask-WTF before anything is saved, because browser-side checks can be bypassed.
- Business rules that span two fields are written in a validate() method of the form class.

Example: ShipmentForm rejects a shipment whose destination equals its origin and any latitude outside −90 to 90.

chainshield-mca/app/forms.py — lines 10-30

```
10 class ShipmentForm(FlaskForm):
11     tracking_no = StringField("Tracking No.", validators=[Optional(), Length(max=30)],
12                               description="Leave blank to auto-generate (CS-xxxx).")
13     origin = SelectField("Origin city", validators=[DataRequired()])
14     destination = SelectField("Destination city", validators=[DataRequired()])
15     transport_mode = SelectField("Transport mode", choices=TRANSPORT_MODES, validators=[DataRequired()])
16     cargo_type_id = SelectField("Cargo type", coerce=int, validators=[DataRequired()])
17     cargo_value = FloatField("Cargo value (₹)", validators=[Optional(), NumberRange(min=0)], default=0)
18     status = SelectField("Status", choices=SHIPMENT_STATUSES, default="In-transit")
19     assigned_operator_id = SelectField("Assigned operator", coerce=int)
20     current_lat = FloatField("Current latitude", validators=[Optional(), NumberRange(min=-90, max=90)],
21                              description="Leave blank to start at the origin hub.")
22     current_lng = FloatField("Current longitude", validators=[Optional(), NumberRange(min=-180, max=180)])
23     submit = SubmitField("Save shipment")
24
25     def validate(self, extra_validators=None):
26         ok = super().validate(extra_validators)
27         if ok and self.origin.data == self.destination.data:
28             self.destination.errors.append("Destination must differ from origin.")
29             return False
30         return ok
```

Screenshot — ShipmentForm in app/forms.py

4.1 Coding Standards (7/9)

7. Security by Default — Role Decorators

- Access rules are written once as decorators and placed on top of each route, so no page can forget its permission check.
- Together with CSRF tokens on every form, Werkzeug password hashing and Jinja2 auto-escaping, this protects against the common web attacks.

Example: @manager_required on /shipments/new returns 403 Forbidden to drivers, pilots, captains and loco pilots.

chainshield-mca/app/blueprints/shipments/routes.py – lines 18-30

```
18 def manager_required(view):
19     """Admins and managers can create, edit and delete shipments."""
20
21     @wraps(view)
22     @login_required
23     def wrapped(*args, **kwargs):
24         if current_user.role not in ("admin", "manager"):
25             abort(403)
26         return view(*args, **kwargs)
27
28     return wrapped
```

Screenshot — manager_required decorator in the shipments blueprint

4.1 Coding Standards (8/9)

8. Template Inheritance and Reusable Macros

- base.html defines the sidebar, top bar and scripts once, and every page only fills its content block.
- Repeated interface pieces are Jinja2 macros, so a design change is made in exactly one place.

chainshield-mca/app/templates/_macros.html — lines 1-7

```
{% macro risk_badge(level, score=None) -%}
  {% if level %}
  <span class="pill risk-{{ level }}">{{ level }}{% if score is not none %} · {{ score }}{% endif %}</span>
  {%- else -%}
  <span class="pill status-pill">Not scanned</span>
  {%- endif %}
{%- endmacro %}
```

Screenshot — risk_badge macro in app/templates/_macros.html

Example: {{ risk_badge(s.risk_level, s.risk_score) }} draws the same coloured risk pill on the dashboard, shipment list and map pop-ups.

4.1 Coding Standards (9/9)

9. Version Control and Pinned Dependencies

- Git records every feature with a descriptive commit message, and every library version is pinned in requirements.txt.
- The Dockerfile installs exactly these versions on Python 3.10, so the application behaves identically on every machine.

Example: Commit “Add admin CRUD, reports with CSV export, demo mode and map zones”; Flask==3.0.3 in requirements.txt.

chainshield-mca/requirements.txt – lines 1-13

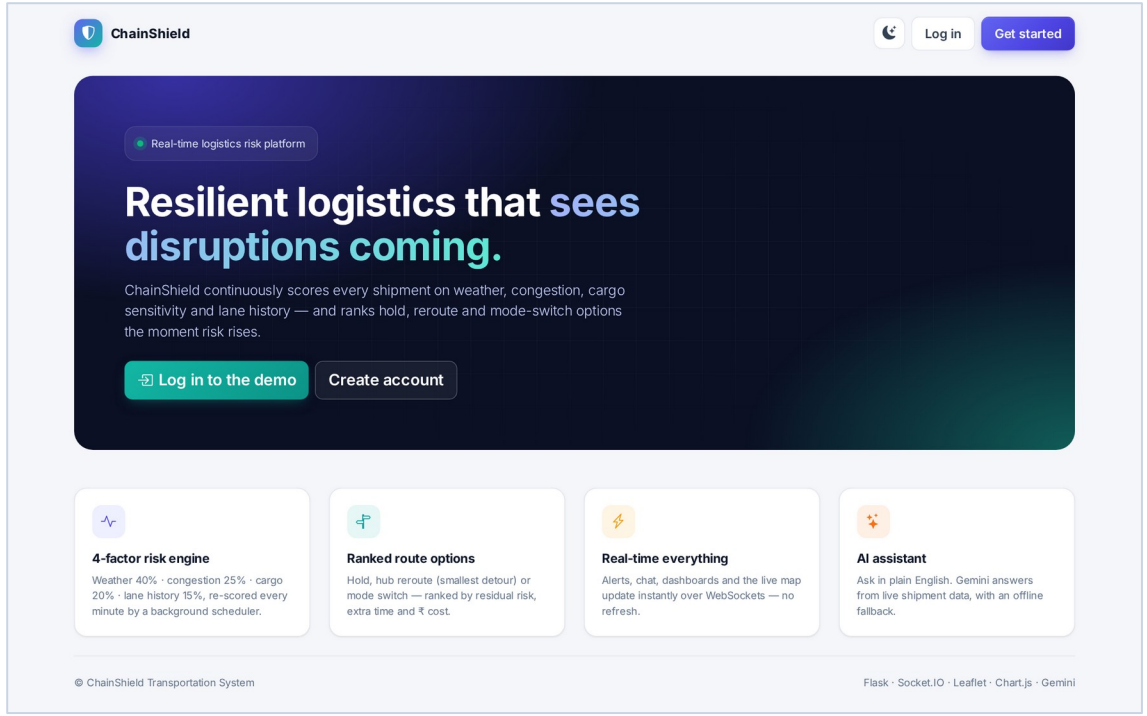
```
1  Flask==3.0.3
2  Flask-SQLAlchemy==3.1.1
3  Flask-Login==0.6.3
4  Flask-SocketIO==5.3.6
5  Flask-WTF==1.2.1
6  APScheduler==3.10.4
7  python-dotenv==1.0.1
8  Werkzeug==3.0.3
9  requests==2.32.3
10 google-generativeai==0.7.2
11 gunicorn==22.0.0
12 eventlet==0.36.1
13 simple-websocket==1.0.0
```

Screenshot — Pinned dependencies in requirements.txt

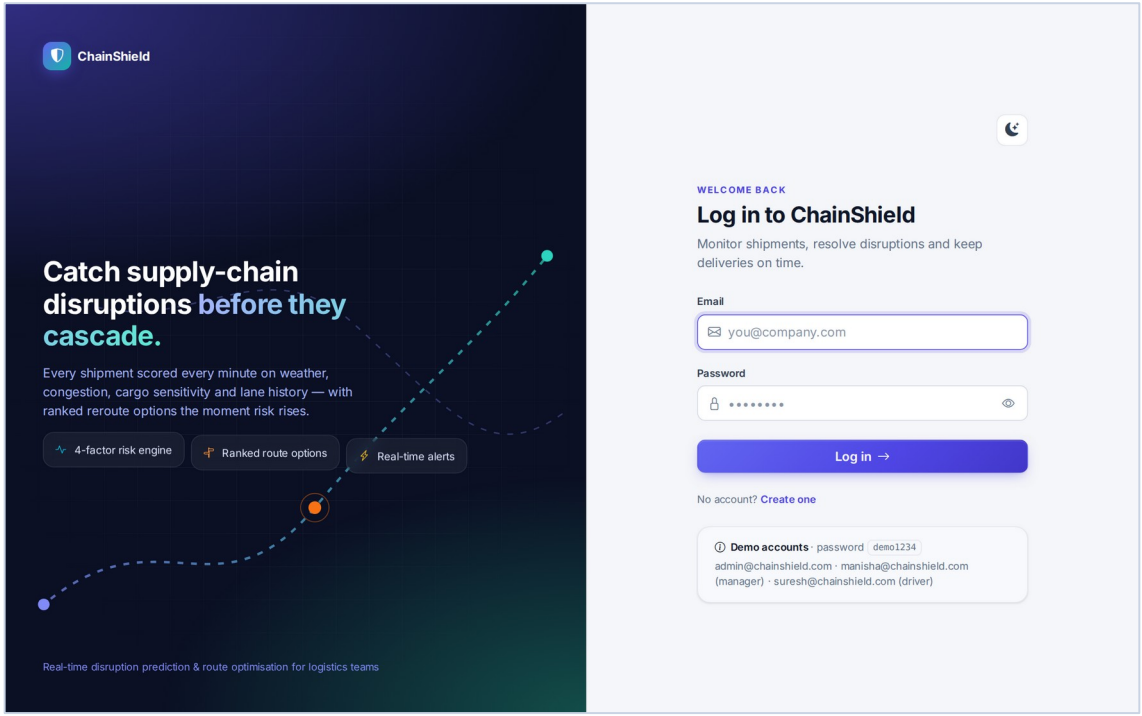
4.2 Implementation Details

- Risk Engine: 0.40 W + 0.25 C + 0.20 Cargo + 0.15 History (each 0–100)
- Thresholds: ≥70 HIGH, ≥50 MEDIUM, ≥30 AT-RISK, else LOW
- Route Solver: hold / continue / 2 hub reroutes / mode switch
- Ranking: 0.60 risk + 0.25 time + 0.15 cost → Option 1 recommended
- Socket.IO personal rooms; live map & charts refresh without polling
- AI Assistant: Gemini on live data, offline fallback
- Security: hashing, CSRF, role checks; Docker + Gunicorn + eventlet

4.3 Output Screens (1/8)

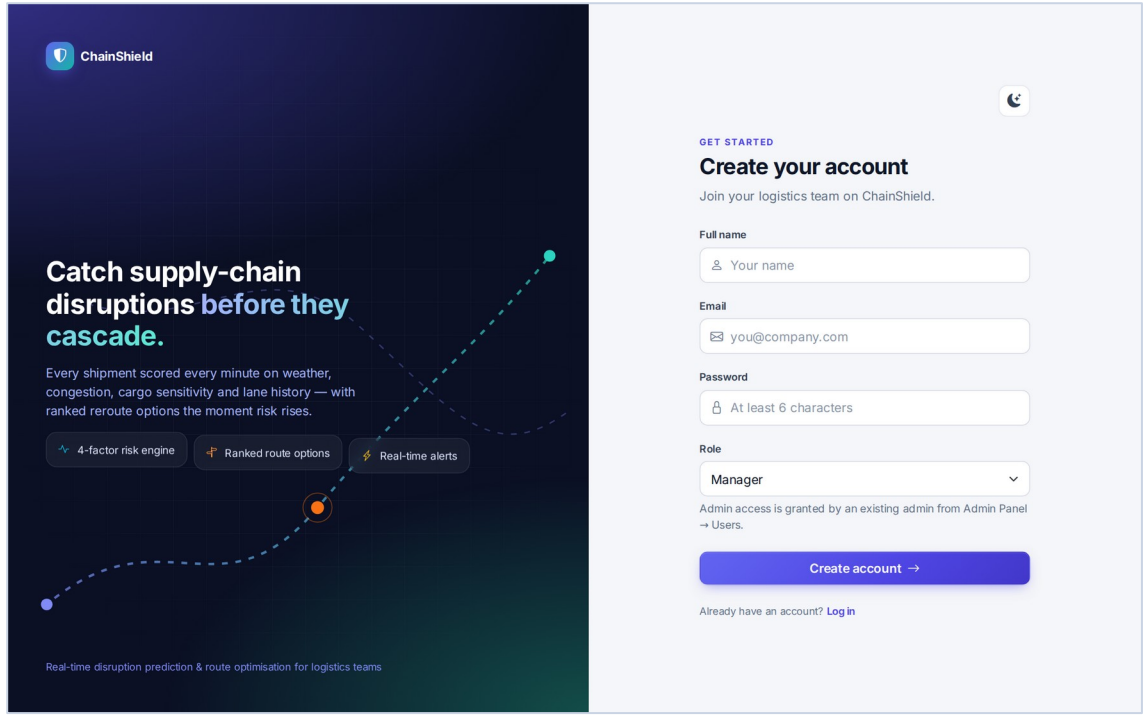


Landing Page



Login Screen

4.3 Output Screens (2/8)

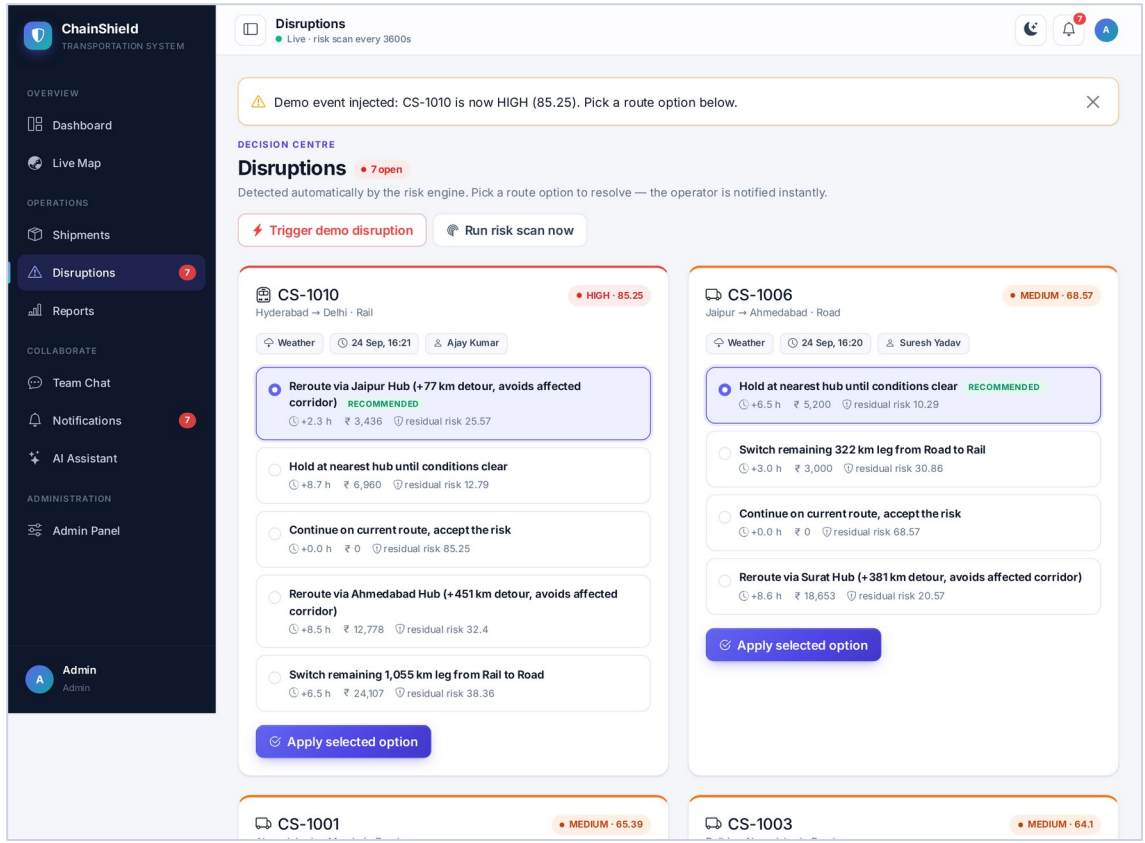


Registration Screen

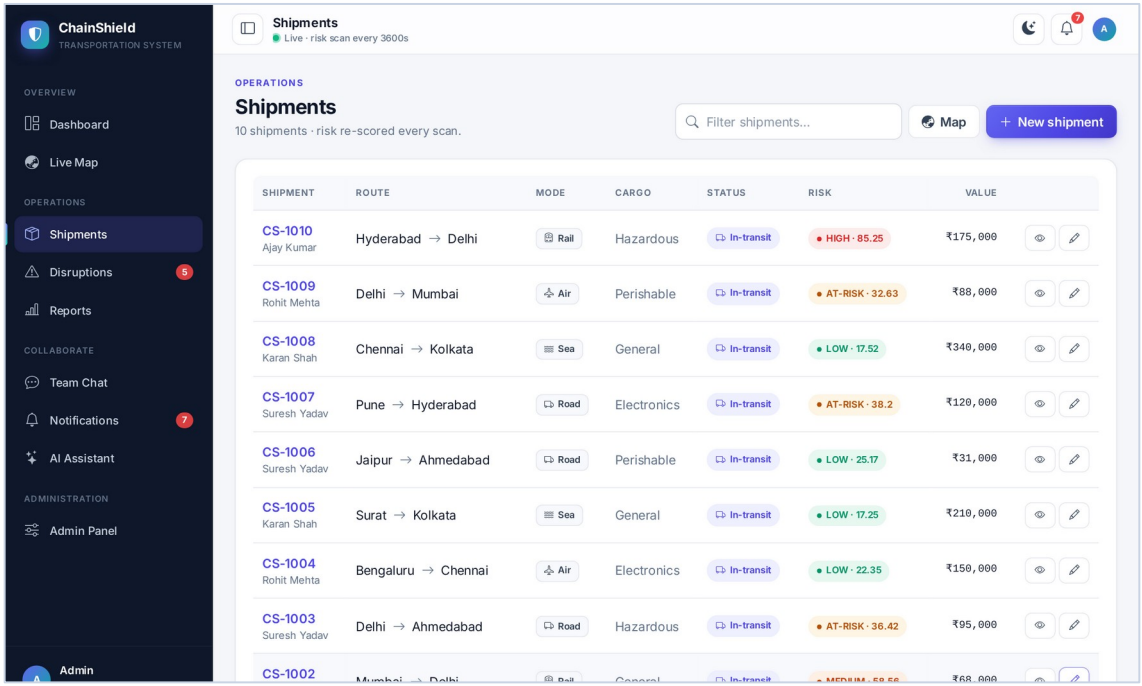


Live Shipment Map with Disruption Zones

4.3 Output Screens (3/8)

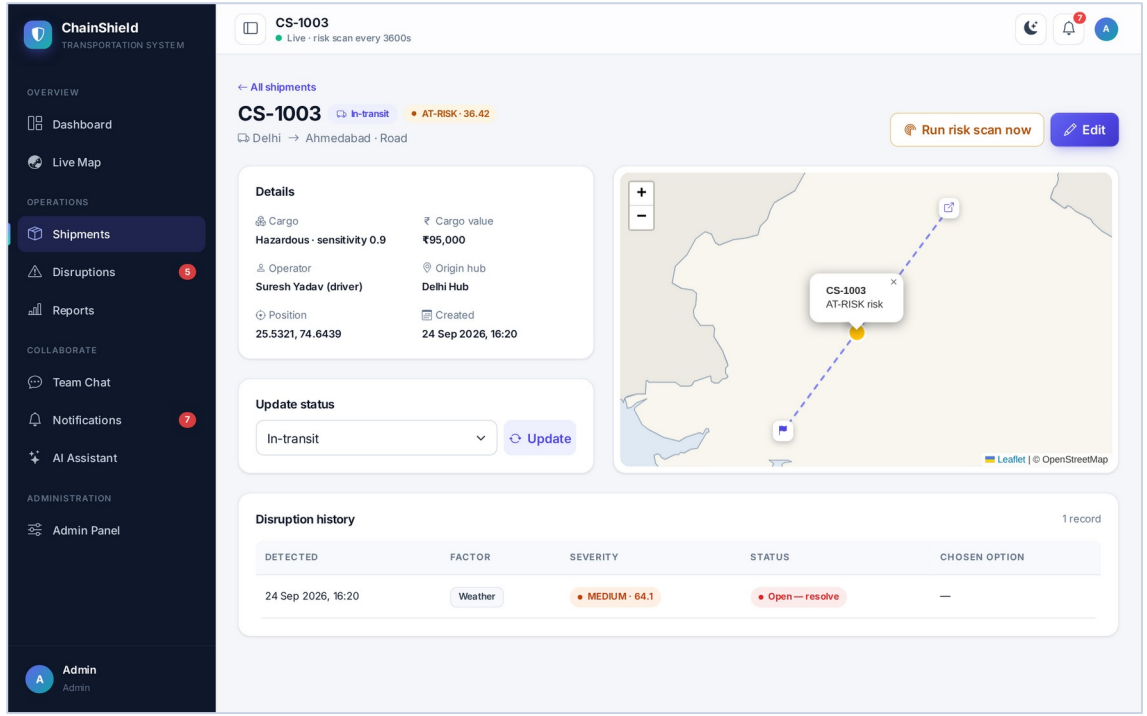


Disruptions with Ranked Route Options

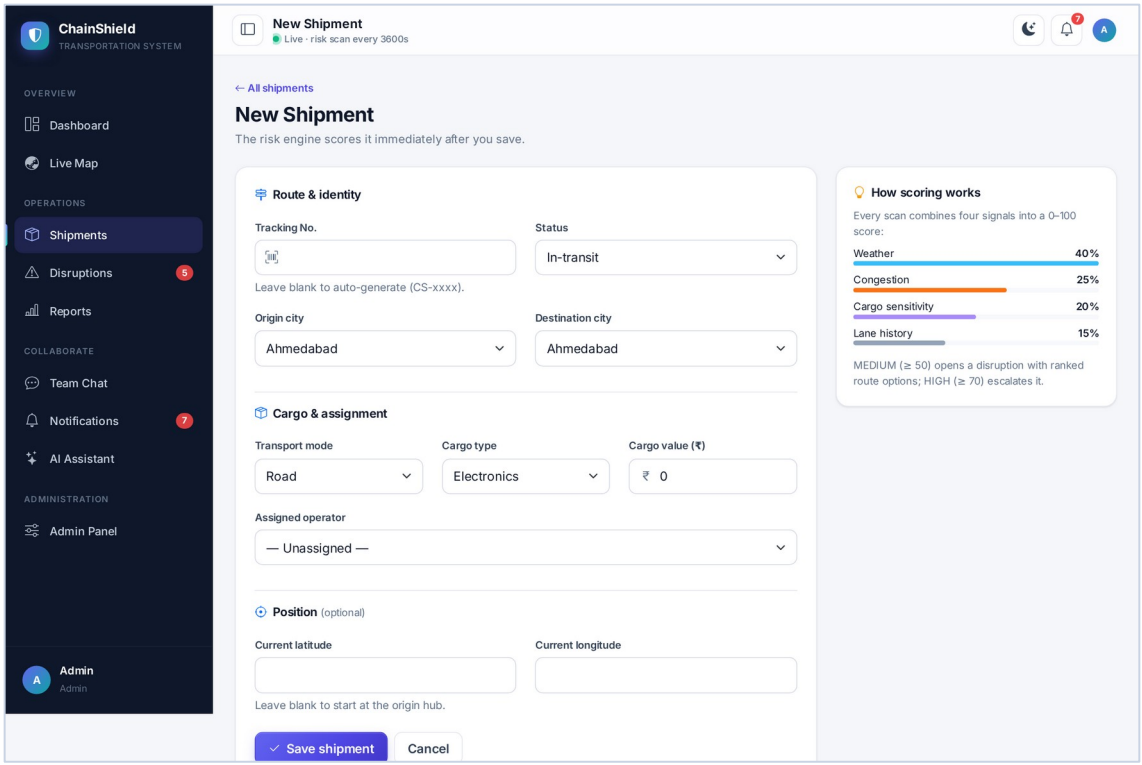


Shipment List

4.3 Output Screens (4/8)

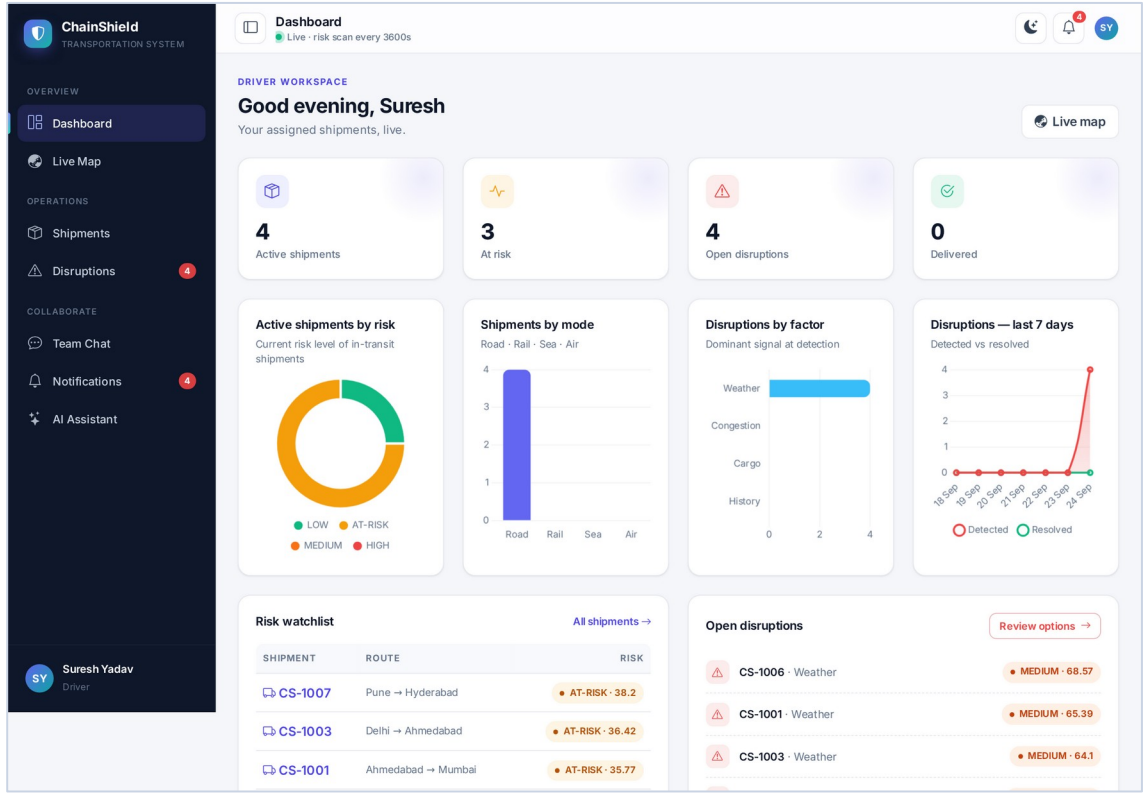


Shipment Detail with Mini-map and History

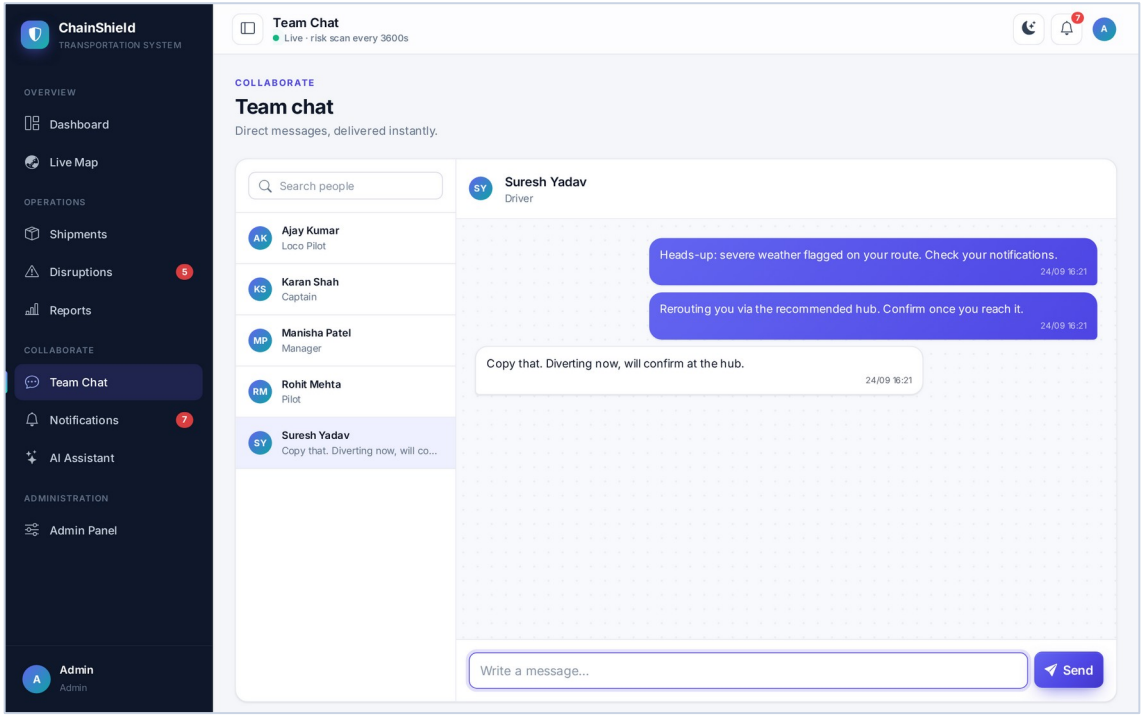


Create Shipment Form

4.3 Output Screens (5/8)

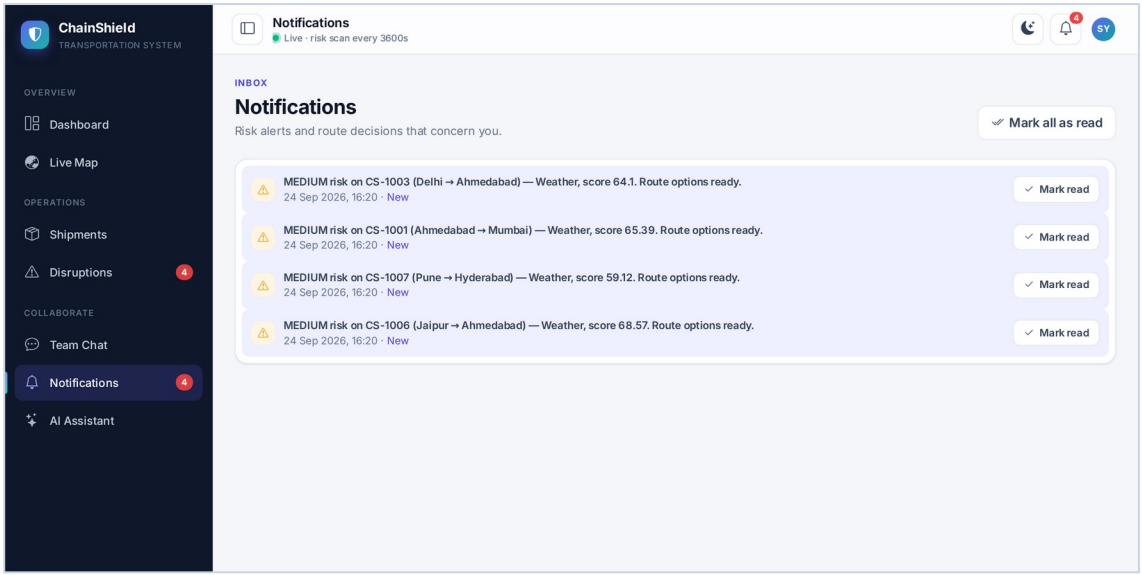


Field Operator (Driver) Dashboard

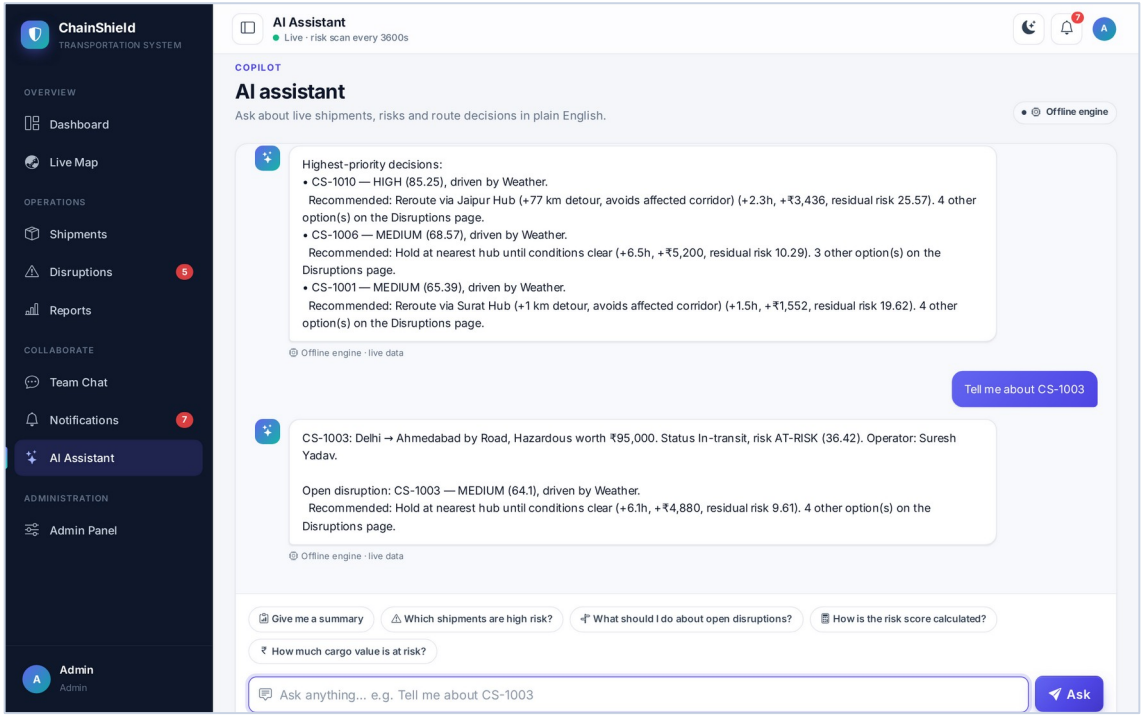


Real-time Team Chat

4.3 Output Screens (6/8)

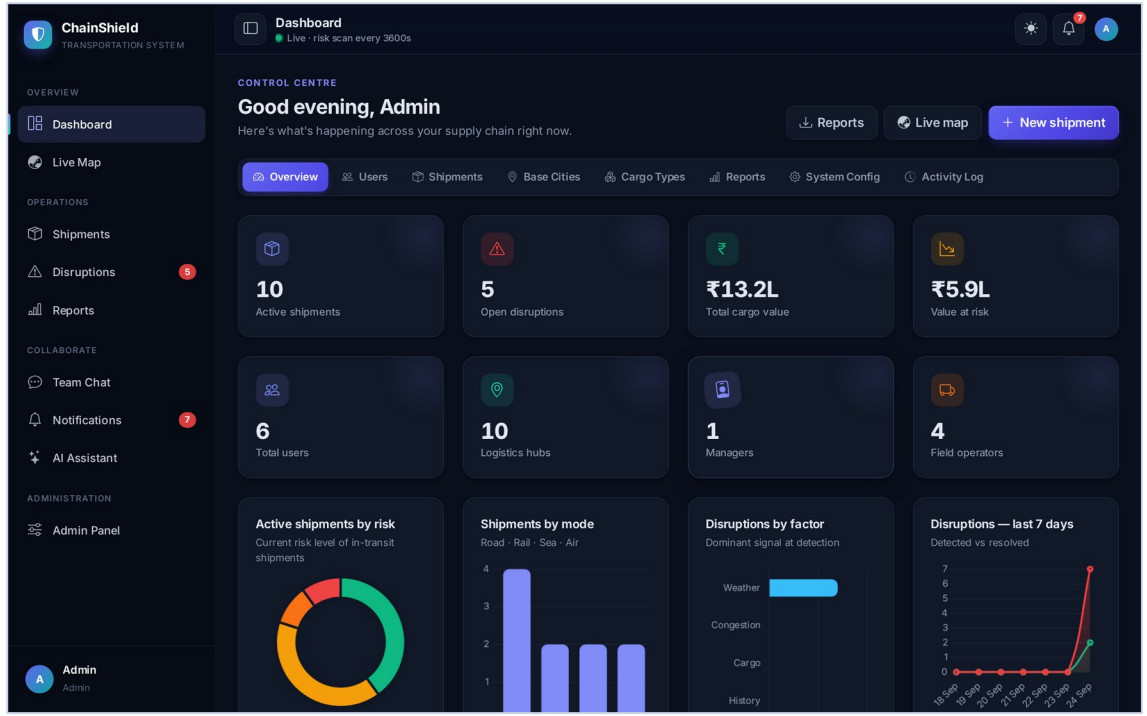


Notification Feed

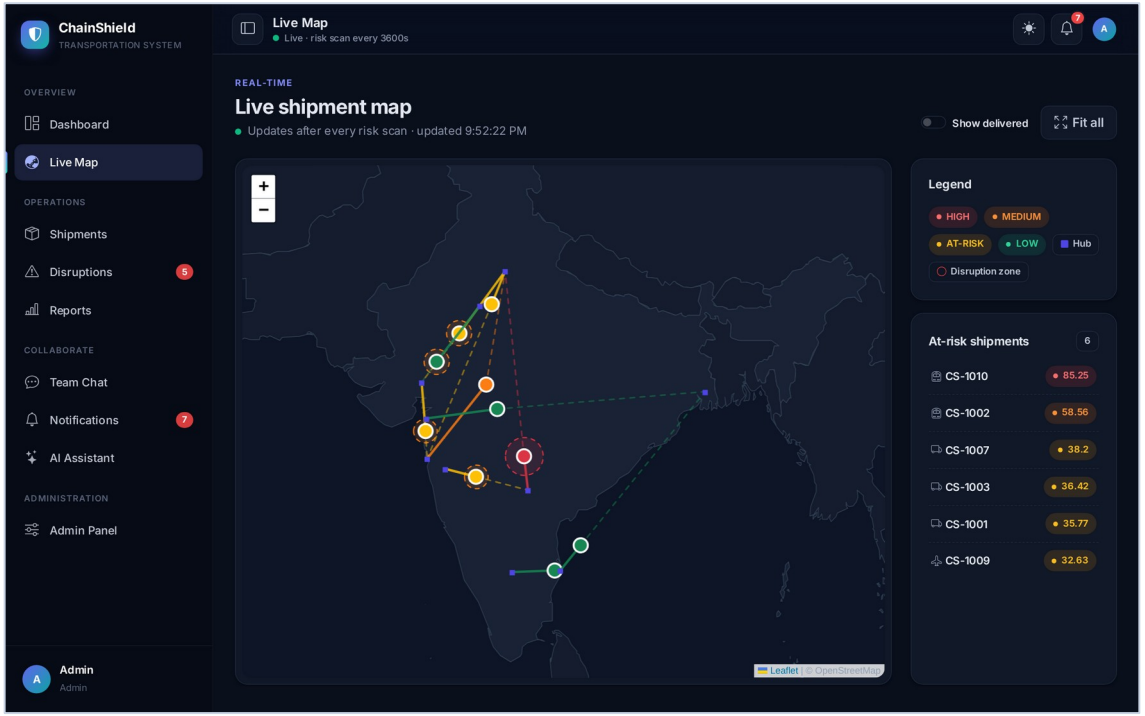


AI Assistant with Suggested Questions

4.3 Output Screens (7/8)

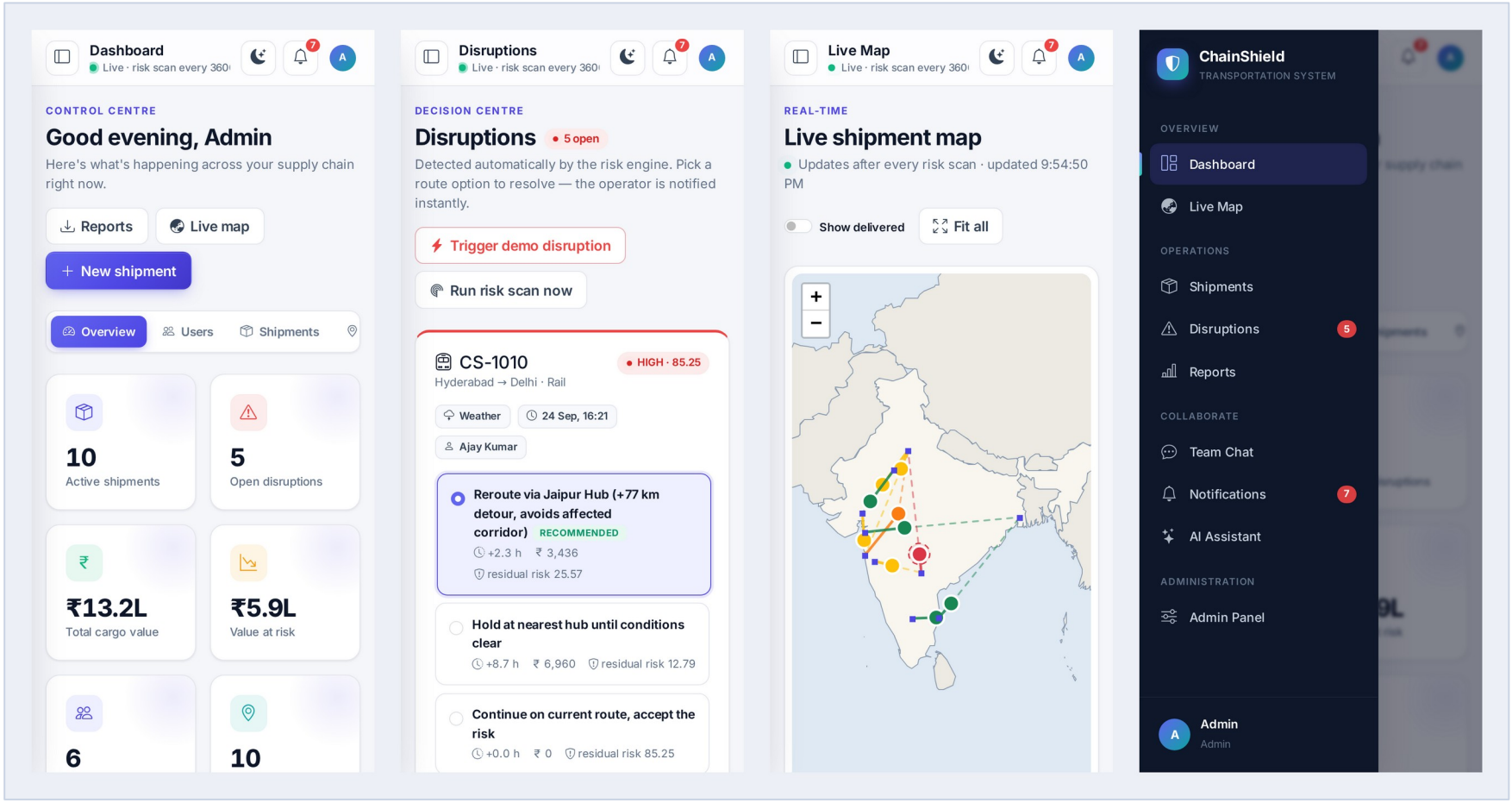


Dark Theme — Dashboard



Dark Theme — Live Map

4.3 Output Screens (8/8)



Mobile Views — Dashboard, Disruptions, Live Map and Navigation Drawer

5.1 Test Cases (1/2) — 101 automated checks, 0 failures

ID	Test Case	Test Data	Expected Result	Status
TC001	User Login	email = manisha@chainshield.com, password = demo1234	User is authenticated and redirected to the role-specific dashboard.	Pass
TC002	Risk Score Calculation	Weather = 80, Congestion = 60, Cargo = 50, History = 40	Score = 0.40×80 + 0.25×60 + 0.20×50 + 0.15×40 = 63 (MEDIUM)	Pass
TC003	Automatic Disruption & Escalation	Scan 1: score 55 (MEDIUM); Scan 2: score 74 (HIGH)	One disruption opened at MEDIUM with ≥ 3 ranked solutions, then escalated to HIGH with re-planned options.	Pass
TC004	Route Solution & Notification	Manager selects Option 1 “Reroute via Jaipur Hub”	Disruption becomes Resolved; operator receives a live notification; action logged.	Pass
TC005	Admin CRUD (Cargo Type)	name = Pharma, weight 0.9 → 0.85; weight 1.5 (invalid)	Cargo type saved, edited and deleted; weight outside 0.0–1.0 rejected.	Pass
TC006	Docker Container Startup	docker compose up --build	Image builds; Gunicorn + eventlet starts on port 5000; /health returns ok.	Pass
TC007	Role-Based Access Control	GET /dashboard/users, /shipments/new, /reports	403 Forbidden for admin/manager pages; 404 for other operators' shipments.	Pass

5.1 Test Cases (2/2) — 101 automated checks, 0 failures

ID	Test Case	Test Data	Expected Result	Status
TC008	Real-time Chat Delivery	Admin sends “Hold CS-1001 at Surat”	Message saved and appears instantly in the driver's open chat.	Pass
TC009	AI Assistant with Fallback	“Give me a summary” with valid and invalid key	Gemini answer with valid key; offline engine answers with invalid key.	Pass
TC010	Reports and CSV Export	/reports?from=2026-09-01&to=2026-09-30; disruptions.csv	Four reports shown; CSV opens in Excel; filter limits rows.	Pass
TC011	User Deactivation	Admin clicks Deactivate	Login refused with message; open session logged out.	Pass
TC012	Demo Disruption Trigger	Click “Trigger demo disruption”	HIGH disruption with ranked options, map zone and live alert appear.	Pass
TC013	Responsive Layout and Themes	Open 8 main pages; toggle dark mode	No sideways scrolling on phone; drawer works; theme remembered.	Pass

6 Proposed Enhancement

- Real GPS / telematics feeds
- PostgreSQL for production scale
- ML-based risk prediction
- Mobile app / PWA for operators
- Multi-tenant support
- SMS & WhatsApp alerts
- Customs & port APIs
- Smarter AI assistant
- PDF reports & scheduled e-mails

7 Conclusion

4	9	101	60 s
Weighted risk factors	Database tables	Automated checks passed	Risk scan interval

- Moves logistics from reactive to preemptive
- Every disruption paired with ranked route options
- Self-hosted Flask + SQLite + Docker
- Fully built, tested and deployable

8 Bibliography

Books

- Flask Web Development — M. Grinberg
- Python Crash Course — E. Matthes
- Fundamentals of Database Systems — Elmasri & Navathe
- Software Engineering — I. Sommerville

Websites

- flask.palletsprojects.com
- docs.python.org
- sqlite.org / docs.docker.com
- getbootstrap.com / leafletjs.com
- flask-socketio / apscheduler docs
- ai.google.dev (Gemini) / openweathermap.org

Thank You

Questions & Discussion

Bhavik A. Visani · Diya Shah · Shruti Somani
Group No. 59 | Guided by Prof. Dhaval Shah | LJ Institute of Computer Application